

Lab 2: Unit Tests

Fall 2026

This lab will have each student create a critter who will be a combatant in a fantasy-style combat game. You will need to develop your critter using test-driven development principles, and then we will bring all the critters together to combat each other in class. May the best critter win!

Notes before getting started:

- You will be provided with three files: `critter.py`, `test_critter.py` and `JaiclinMarcus.py` (you'll rename this one).
- You **should not modify** `critter.py`! At all. Read it, but don't edit it.
- You should rename `JaiclinMarcus.py`. Your filename should be: `[YourLastName][YourFirstName].py`, for example: `ChenJaiden.py`. You will need to modify this file to provide the attributes and methods of your particular critter.
- Your class must inherit from the provided Critter class.
- You are not allowed to override any of the attribute validation that is provided in the Critter class.

How to get started (we will do this part together in class):

- Open your class file, the one with your name as its filename. Change the name of the class on line 3 to the name you wish to use.
- Open `test_critter.py`, change lines 3 & 6 to match your class name & file name.
- Open `critter.py`, look at the docstring for the `Critter` class (lines 17-29). This lists the nine methods you will need to implement or override in the file with your name as its filename. I've provided stubs for all of them.
- Notice, at the bottom of `critter.py`, I've provided three convenience functions to make your coding easier.
- Run the unit tests and make sure that all tests pass. Once this passes, you are now ready to develop.
- I've provided some things in `critter.py` for you to use in your development:
 - A distance function, so you can know how far away anyone else on the field is.
 - I use a direction number 1-8 for moves and attacks (See lines 111-113 of `critter.py`). But, sometimes you'll need to use xy-coordinates instead. For example, to your left is both direction #5, and (-1,0) from where you are. So, I provide conversion functions `direction_to_xy()` and `xy_to_direction()` to make it easy for you to go back and forth.
 - Check out lines 51-68, this section of the `__init__()` method gives you some attributes that will help your character make decisions. The values of these are

all automatically set by the game engine as the game plays out. Notice that you can access these values for both your own critter and your opponent's critter.

- Notice that, in your `close_attack()`, `far_attack()`, and `move()` methods, one of the inputs is a list of critters. So, in these methods, you have access to a list of everyone on the field, and you can use that to access the current values of your enemies' attributes. But, remember that your critter is in this list too, so make sure you check which one is you and look at the other one for information about the enemy.

How to proceed:

- Pick one of the nine methods that you want to develop. ***Do not start by writing the method, start by writing the unit test.*** Write the unit test for the method, then write the method. You can do this incrementally: write a unit test for one bit, then write that bit, then run the test, repeat.
 - For the methods that just return a string, this will be very straightforward.
 - For each of the attacks, this will be a bit more complicated:
 - For move, this will be quite a lot more complicated. Maybe do this one last.
- To write a unit test for move or for an attack, you should write out code that creates a scenario, for ex: My critter is at (x1, y1), there's an enemy (generic `Critter`, pick some stats) at (x2, y2), the enemy has only a close attack, so I expect it to move straight towards my critter, what should my response be?
- Now, vary your enemy's location and/or your own, or your/its health or other attributes, decide what your critter's reactions should be. Test that this reaction is returned. Then, another scenario, then another, for several different possibilities.
- Remember that you can access all of the attributes of your enemy as you code your critter's actions.
- For the first round of this Lab, we will use only one-on-one combat.
- If your critter has only `close_attack_power` nonzero or only `far_attack_power` nonzero, you do not need to implement anything for unit tests or functionality for the other type of attack. The other attack method will automatically return `None`. I expect you to implement at least one attack and its associated unit tests.
- You can feel encouraged to raise `ValueErrors` in your code. The game engine will catch any that occur in the course of the game. Make sure you only use `ValueError`, however.

Some expectations you can have about how the game will work:

- Your critter will have to act entirely autonomously for the entire combat, based only on the return values from the three action methods (`move`, `close_attack`, `far_attack`).
- Your critter's health is your life points. When your critter is hit by an attack, its health will go down. When your critter's health is 0 or less, your critter will die.
- Your critter will get one attack per turn (either close or far, not one of each). It is possible that your critter will have to pass on making an attack if no enemy is in range through the course of a round. If you request a second attack in a turn, it will automatically be turned into a pass. Notice that there is an attribute called `used_attack` to help you detect if

you've attacked – it returns `True` if you have used your attack in this round, and `False` if you have not. You can use this to avoid calling for a second attack in a round.

- You can only close attack an enemy that is adjacent to you (one of the 8 squares in the move description).
- You cannot far attack an enemy that is adjacent to you. You can only far attack if $2 \leq \text{distance} \leq \text{far_attack_range}$.
- Your `defense` value will affect **both** how hard it is for an attack to successfully hit you, **and** how much damage you will take.
- Your `close_attack_power` will affect **both** how likely you are to hit with a close attack, **and** how much damage you do, and same for `far_attack_power` for far attacks.
- If, in an attack, `power == defense`, then you should expect that the attack should succeed about 50% of the time and, if the attack is successful, then damage will be, on average, 0.5 health points per point of attack power.
- Your `speed` is the number of squares you can move per round.
- You should expect that, in one round, your `move()` method will get called `self.speed + 1` times, one of which is expected to return an attack. If your `move()` method returns `0`, you will not get this opportunity to act back later in the turn, that action is lost.
- If the battle lasts 20 rounds, then the battle will end as a tie. Scoring will be Win: 3pts, Tie: 1 pt, Loss: 0 pts. So, playing for a tie is not necessarily irrational once your health gets low.
- The board is 10x10. The values of x & y range from 0 to 9. Any value of x or y that is below 0 or above 9 is out of range.